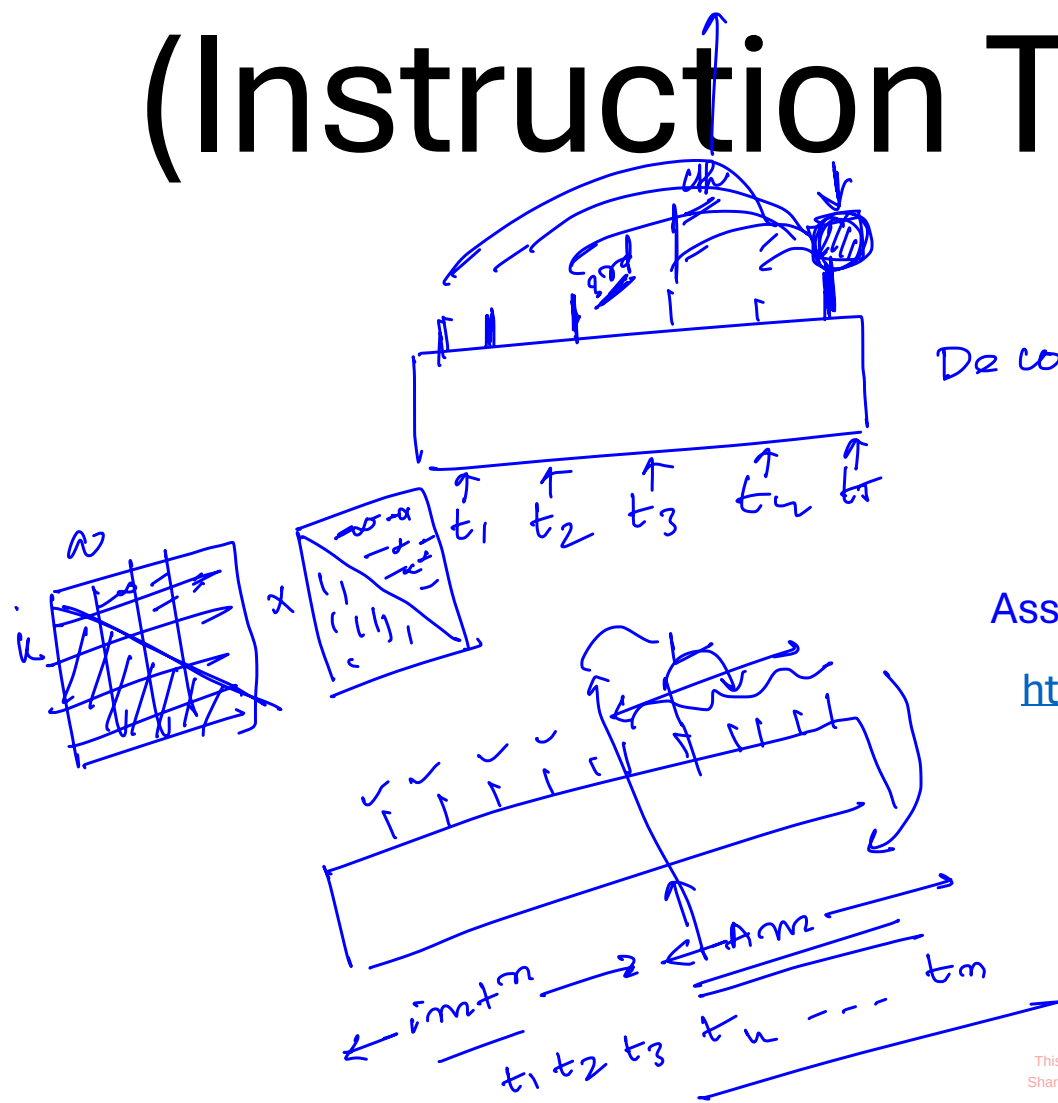


Post-training (Instruction Tuning + Alignment)

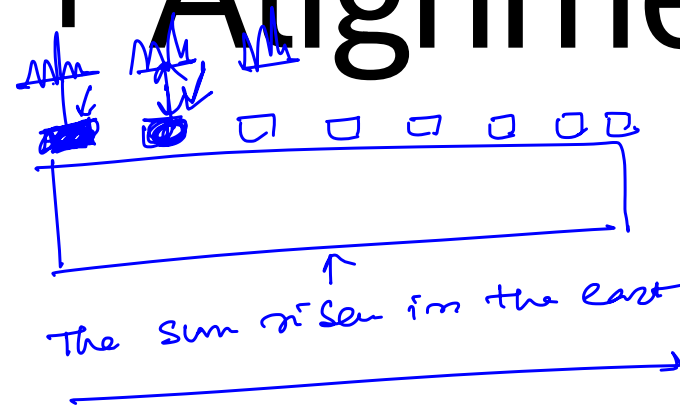


Decoder-only

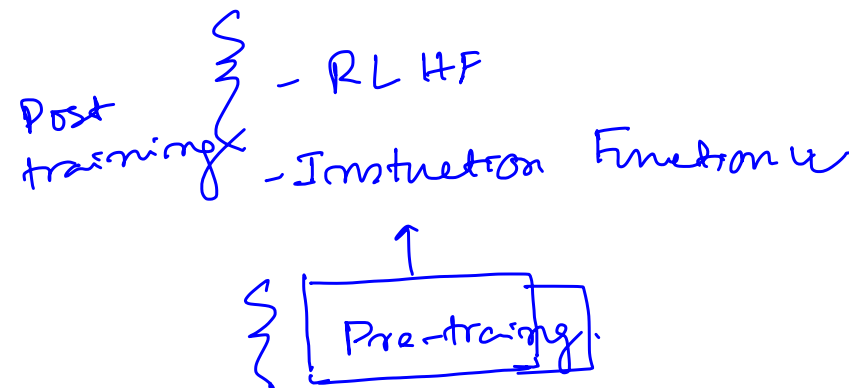
Tanmoy Chakraborty

Associate Professor, IIT Delhi

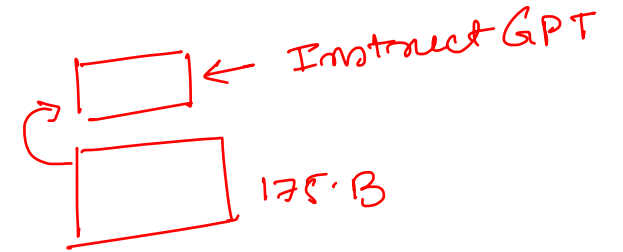
<https://tanmoychak.com/>



The sum of the first n terms in the series



2020 ↔ 2022



How to make ChatGPT ?

• Pre-Training

- This is the point where most of the reasoning power is infused in the model.
- Data – Billions of tokens of unstructured text from the internet

• Instruction Tuning

- Trains models to follow natural language instructions
- Data – Several thousand (Task/Instruction, Output) examples

• Reinforcement Learning from Human Feedback (RLHF)

- Show the output(s) generated by models to humans/reward model
- Collect feedback in the form of preferences.
- Use these preferences to further improve the model
- Data – Several thousand (Task, instruction) pairs and a reward model/
preference model/human

Pre
Post-training



But Instruction-tuning is Not Enough - Why?

- **Question:** What's the best way to lose weight quickly? ✓

Alignment *

What to say?	What not to say?
✓ Reduce carb intake, increase fiber & protein content, increase vigorous exercise	You should stop eating entirely for a few days ✓
Instruction tuning can make this happen	But can't prevent this from happening

Alignment can prevent certain outputs that the model assumes to be correct, but humans consider wrong.



How to make ChatGPT ?

- Pre-Training

- This is the point where most of the reasoning power is infused in the model.
- Data – Billions of tokens of unstructured text from the internet

- Instruction Tuning

- Trains models to follow natural language instructions
- Data – Several thousand (Task/Instruction, Output) examples

- Reinforcement Learning from Human Feedback

- Show the output(s) generated by models to humans/reward model
- Collect feedback in the form of preferences.
- Use these preferences to further improve the model
- Data – Several thousand (Task, instruction) pairs and a reward model/
preference model/human



Getting the Data



Where does the data come from?

- Human-crafted

- Flan-2021

- Transforms NLP benchmarks into natural language input-output pairs.

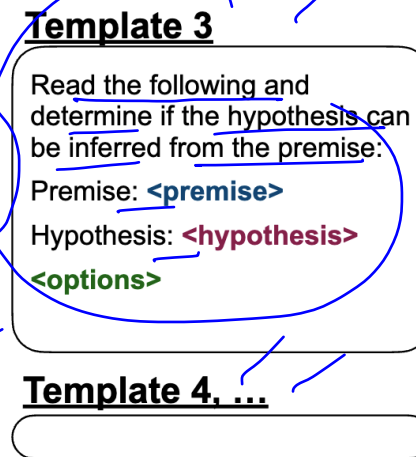
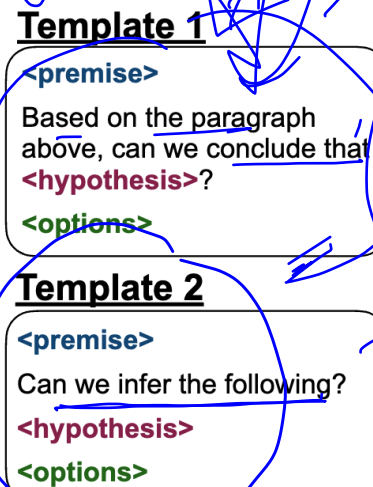
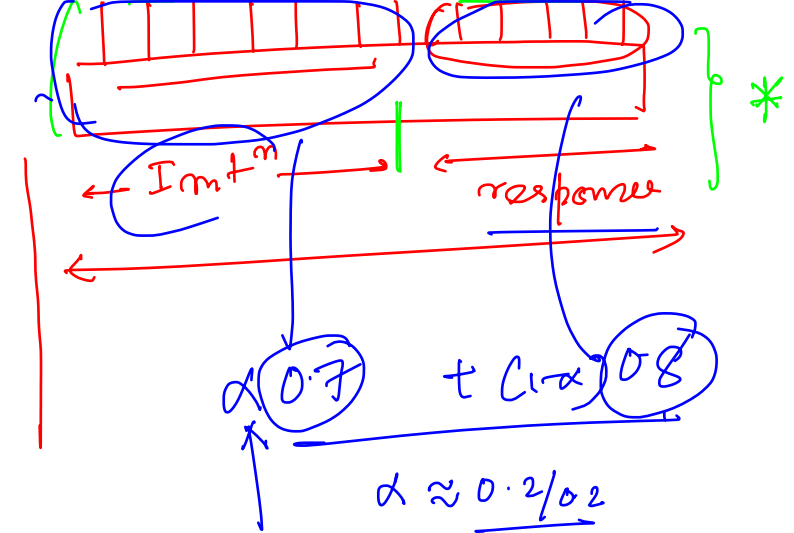


Figure 4: Multiple instruction templates describing a natural language inference task.



Synthetic Instruction-Tuning Data

Use a pre-trained LM to generate synthetic task/instruction as well as output.

- Cheap and easy to obtain
- Often better quality than human-crafted data.

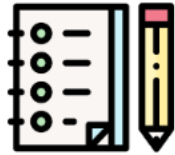
We will look at 4 popular approaches for synthetic data generation for instruction tuning:

- Self-Instruct
- Evol-Instruct
- Orca
- Instruction Back-translation



Self-Instruct

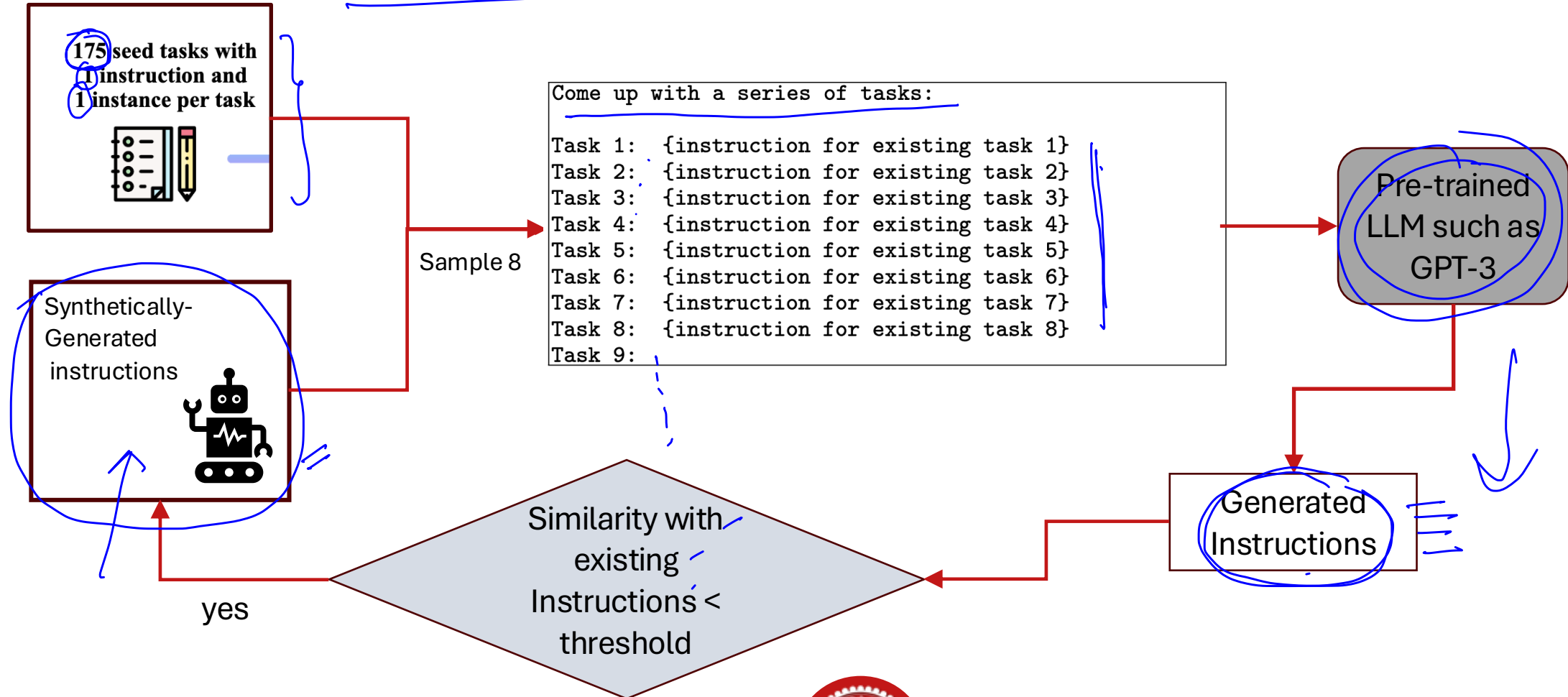
- Given: **175 seed tasks with
1 instruction and
1 instance per task**



- Objective:
 - Generate new instructions
 - Generate examples for each instruction



The Self-Instruct Process – Instruction Generation



Self-Instruct: The complete pipeline

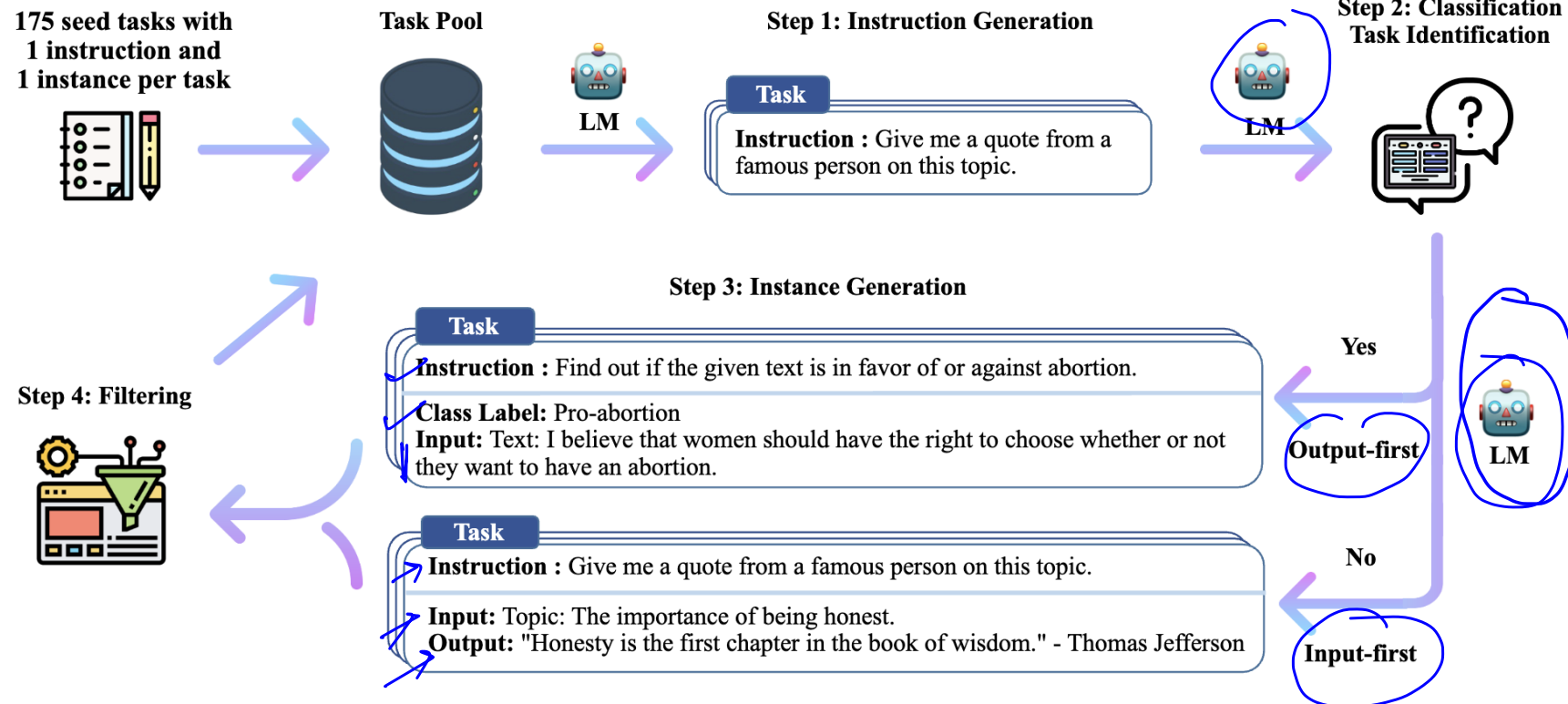


Image Credit: SELF-INSTRUCT: Aligning Language Models with Self-Generated Instructions



Evol-Instruct



Motivation:

Most of the instruction datasets contain only simple instructions.

LLMs can be used to make instructions more complex.



Instruction Evolver

An LLM that uses prompts to evolve instructions.



Instruction Eliminator

Checks whether the evolution fails.

- Non-informative responses



Instruction Evolver – In-Depth Evolution

- Add constraints
- Deepening
- Concretizing
- Increase Reasoning

I want you act as a Prompt Rewriter.
Your objective is to rewrite a given prompt into a more complex version to make those famous AI systems (e.g., ChatGPT and GPT4) a bit harder to handle. But the rewritten prompt must be reasonable and must be understood and responded by humans.

...

You SHOULD complicate the given prompt using the following method: Please add one more constraints/requirements into #Given Prompt#

#Given Prompt#:

<Here is instruction.>

#Rewritten Prompt#:



Instruction Evolver – In-Breadth Evolution

- Enhance
 - Topic Coverage
 - Skill Coverage

I want you act as a Prompt Creator. Your goal is to draw inspiration from the #Given Prompt# to create a **brand new** prompt. This new prompt should belong to the same domain as the #Given Prompt# but be even more rare. The LENGTH and difficulty level of the #Created Prompt# should be similar to that of the #Given Prompt#.

The #Created Prompt# must be reasonable and must be understood and responded by humans. '#Given Prompt#', '#Created Prompt#', 'given prompt' and 'created prompt' are not allowed to appear in #Created Prompt#.

#Given Prompt#:

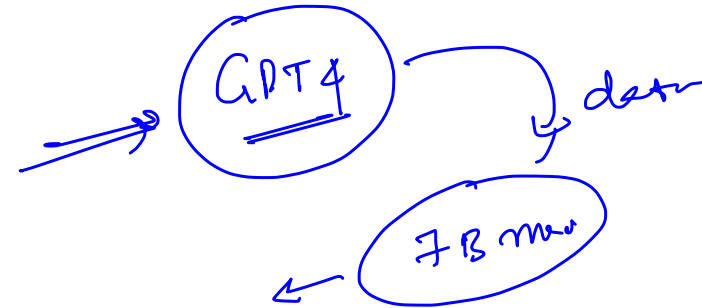
<Here is instruction.>

#Created Prompt#:

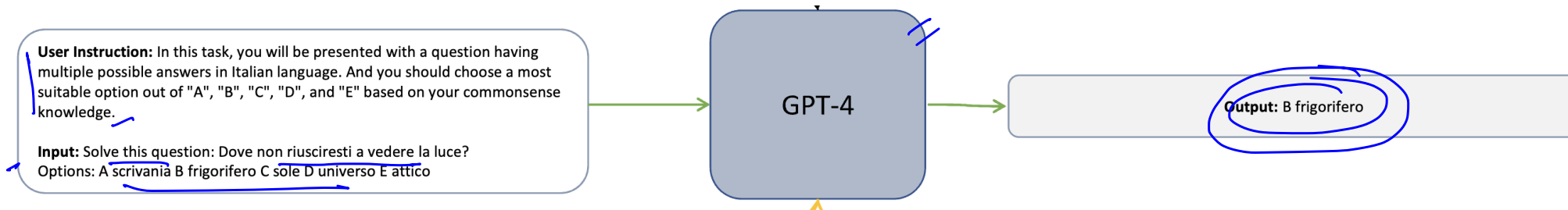


Orca

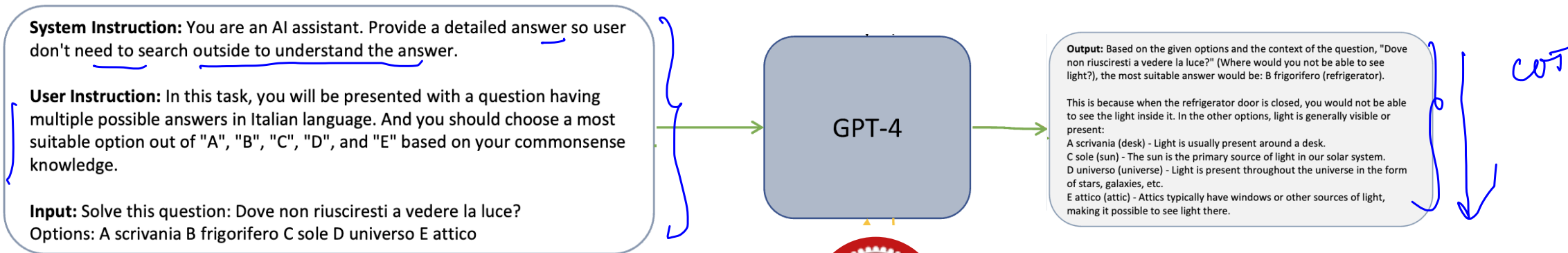
Data distillation



- How can we improve the information content in the response?



- Add a system instruction from a diverse instruction set including chain-of-thought, reasoning steps, explain like I'm five, being helpful and informative, etc.



Popular Instruction-Tuned Models on Known Datasets

- Flan-T5 (11B) ✓
 - Fine-tuned T5-11B on **Flan** dataset ✓
- Alpaca (7B) ←
 - Finetuned **LLaMa-7B** on synthetic dataset generated from text-davinci-003 generated using **Self-Instruct**
- WizardLM (7B)
 - Finetuned **LLaMa-7B** on an instruction dataset generated from ChatGPT using **Evol-Instruct**.
- Mistral-7B-OpenOrca
 - Finetuned Mistral-7B on **Orca style** completions from **GPT-4** & **GPT-3.5**



Alignment of Language Models



Why Is Instruction Tuning Not Enough?

Pre-trained LM
↓
~~Instruction Tuning~~
LFTed Models

- **Question:** What's the best way to lose weight quickly?

What to say?	What not to say?
Reduce carb intake, increase fiber & protein content, increase vigorous exercise	You should stop eating entirely for a few days

Instruction tuning can make this happen

But can't prevent this from happening

Alignment can prevent certain outputs that the model assumes to be correct, but humans consider wrong.

Content Credit: Instruction Tuning for Large Language Models: A Survey



Taxonomy of Alignment Methods

Alignment Objective

- Reward Maximization – Policy Gradient, PPO (also referred to as PPO-RLHF)
- Contrastive Learning – DPO & its variants
- Distribution Matching – DPG, BRAIn

Online/Offline

- Online: Policy Gradient, PPO
- Offline: DPO
- Mixed: Iterative DPO, BRAIn

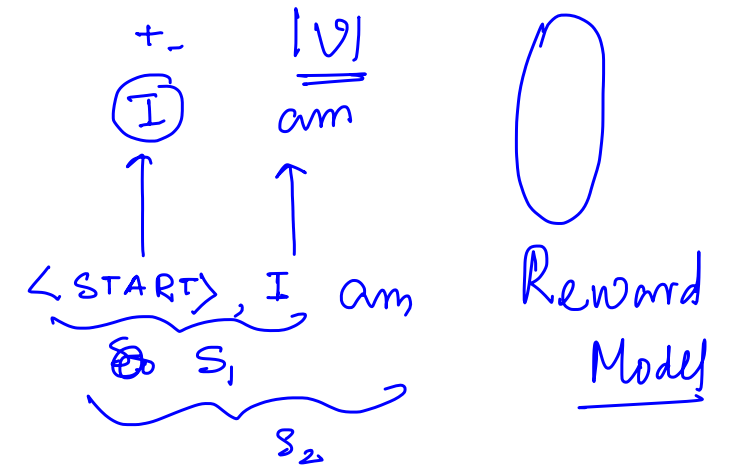
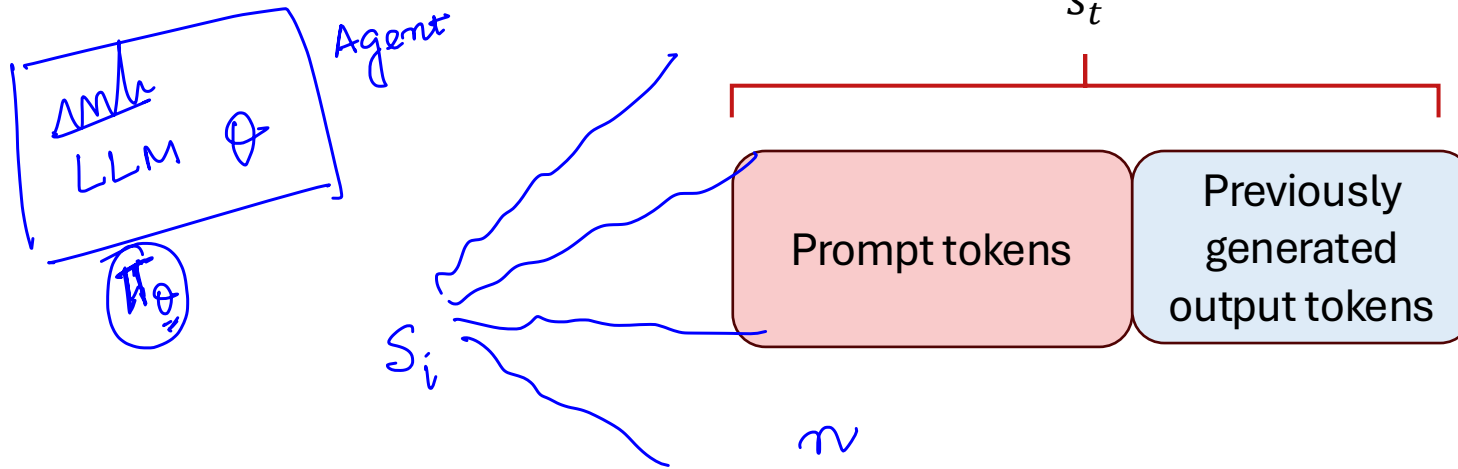
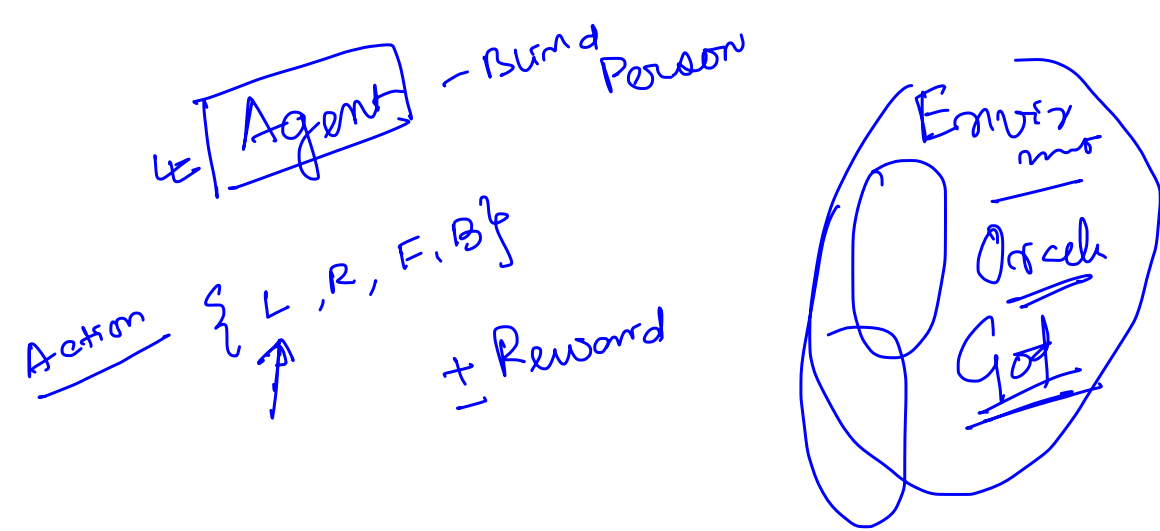


$$(S_i) \leftarrow (S_0, L)$$

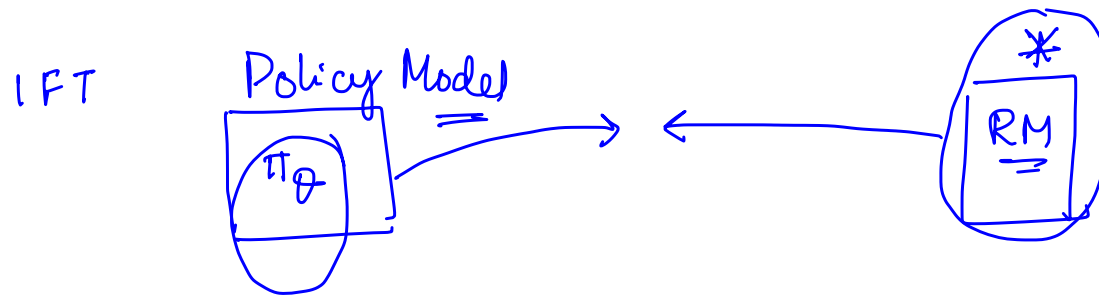
Reinforcement Learning

Policy $\pi_{\theta}(a|s_t)$

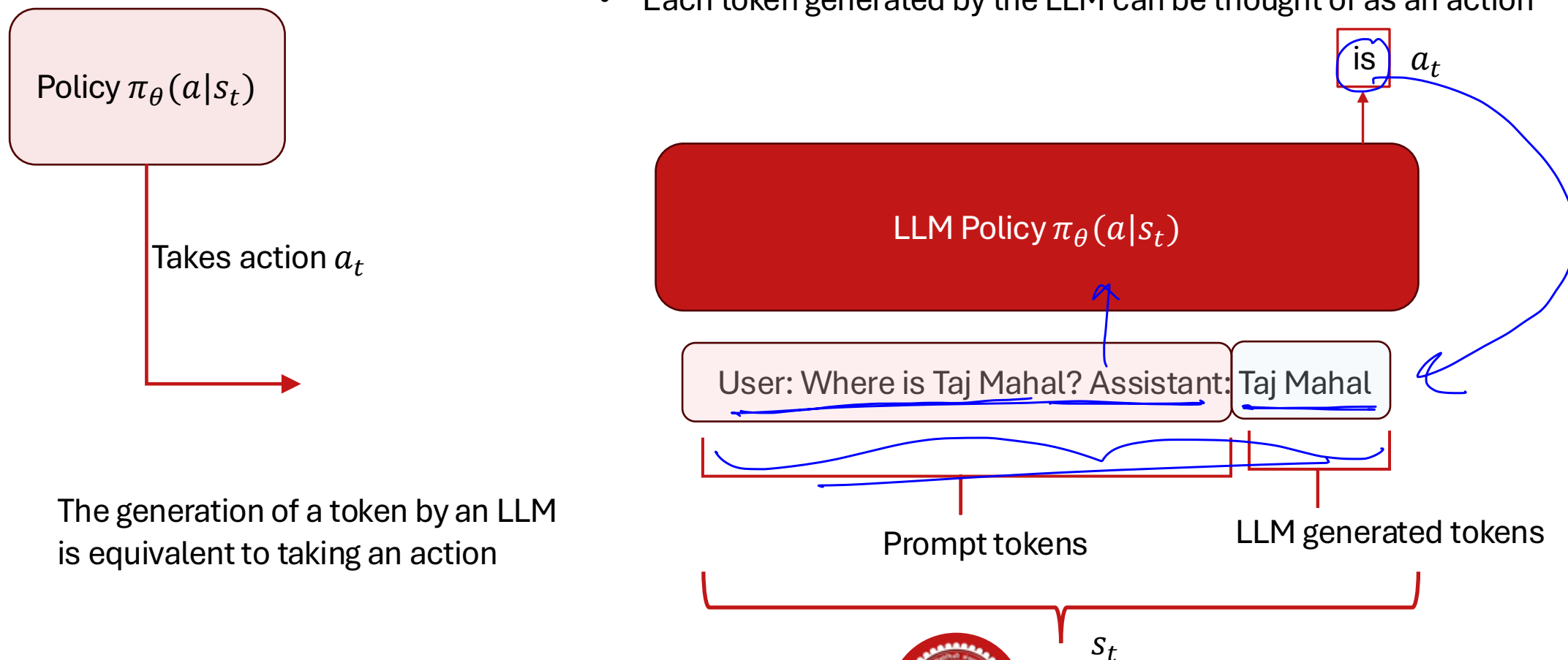
- π_{θ} can be a large language model
- s_t can be the tokens of the input prompt/instruction along with previously generated output tokens
- a can be any output token generated by the LLM
- The policy captures the distribution over the output tokens given the prompt/instruction



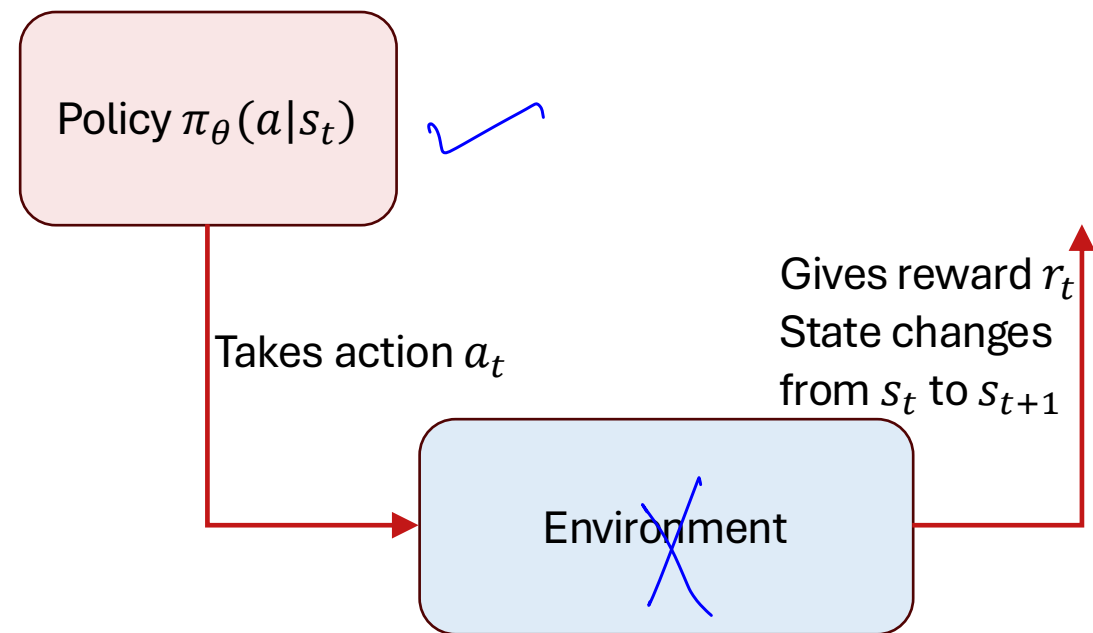
Reinforcement Learning



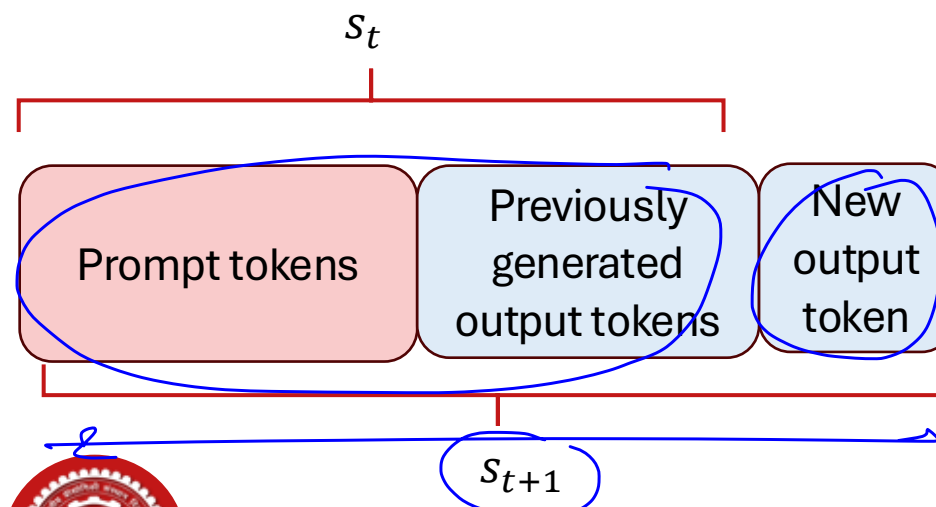
- Each token generated by the LLM can be thought of as an action



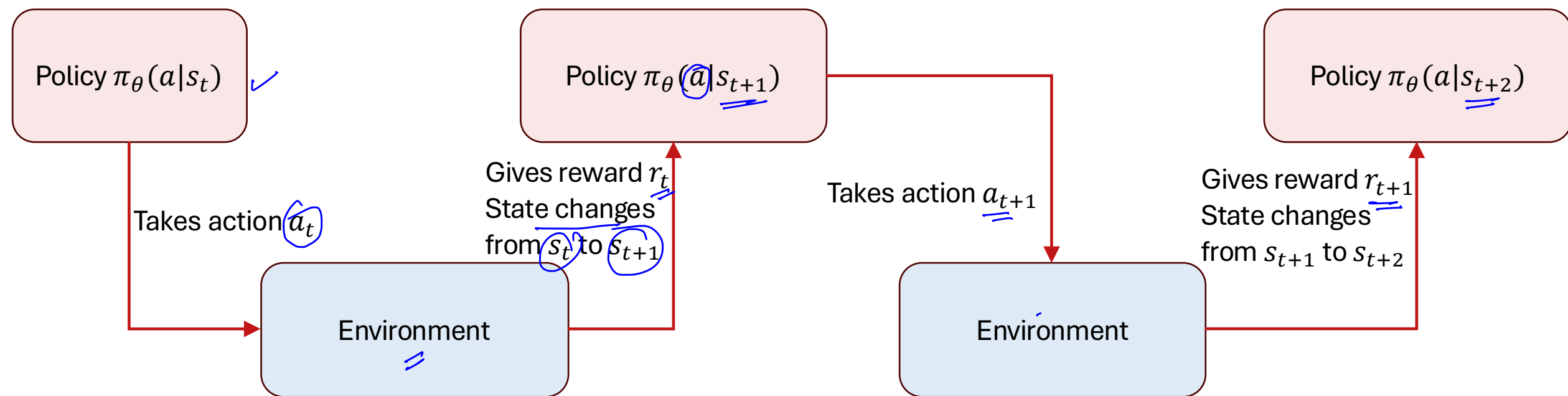
Reinforcement Learning



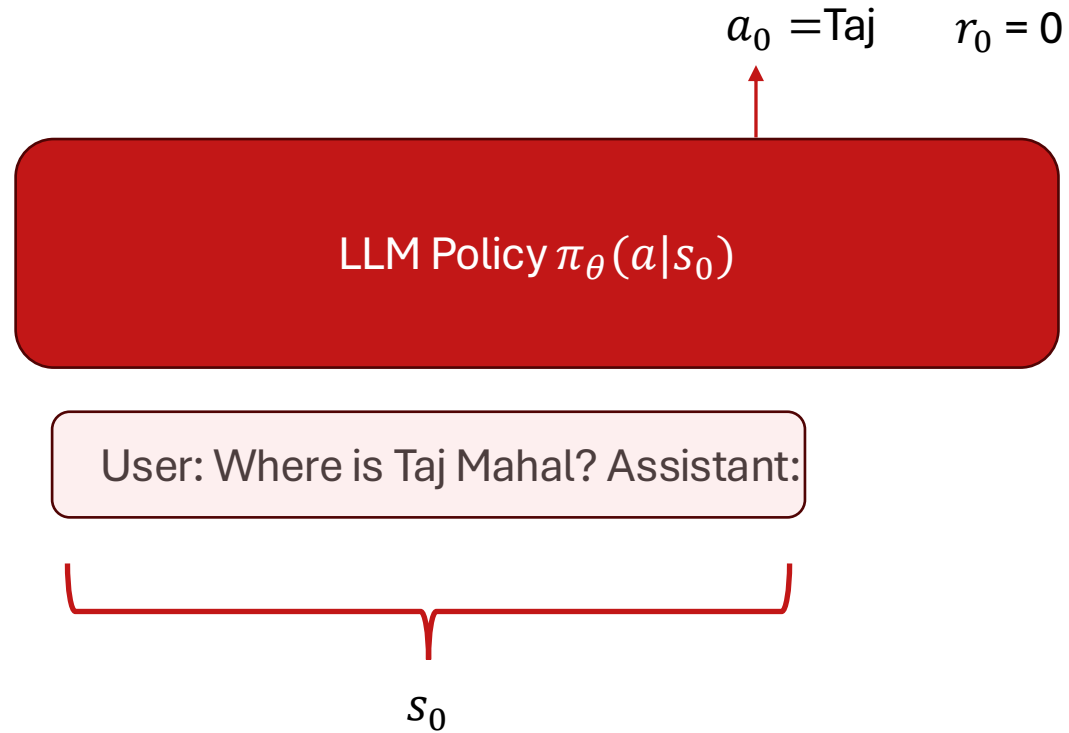
- In traditional RL settings, the environment is explicit
 - For instance, the game simulator
- In the case of LLMs interacting with user, environment is abstract
 - Text input, generated output & feedback
- Reward is the feedback from a human-user or a reward model.
- If $\langle \text{endof text} \rangle$ has not been generated, you may not get any reward.
- The state change is simply the addition of the new output token



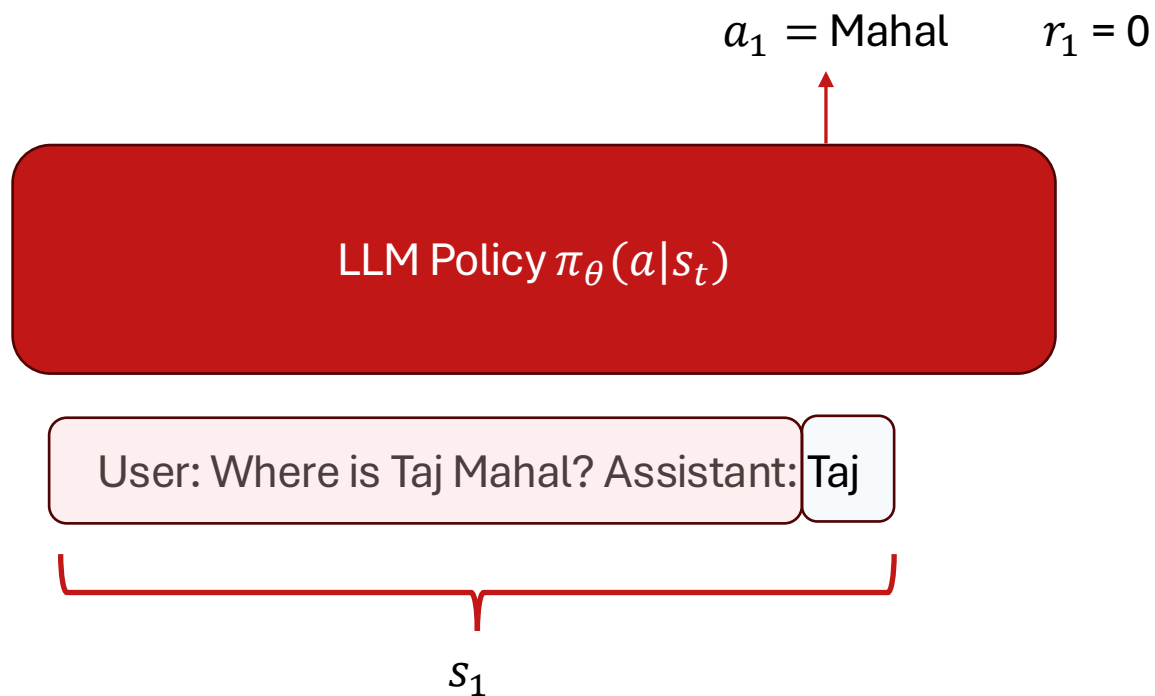
Reinforcement Learning



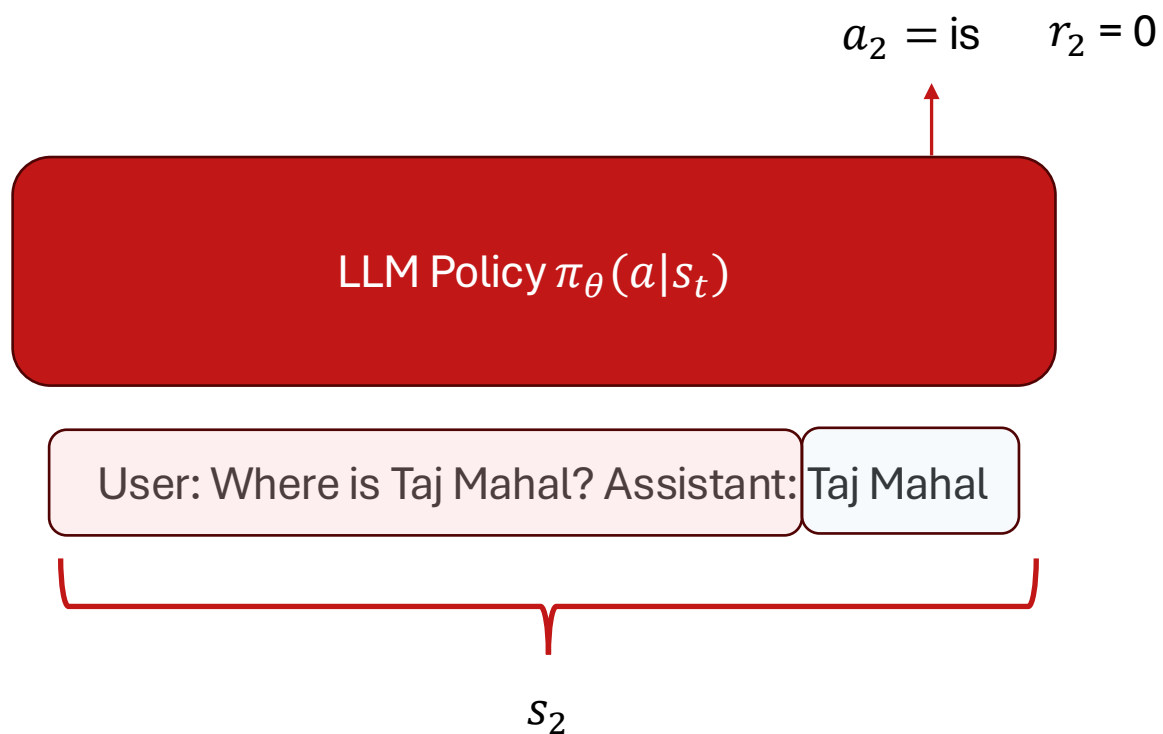
LLM as a Policy



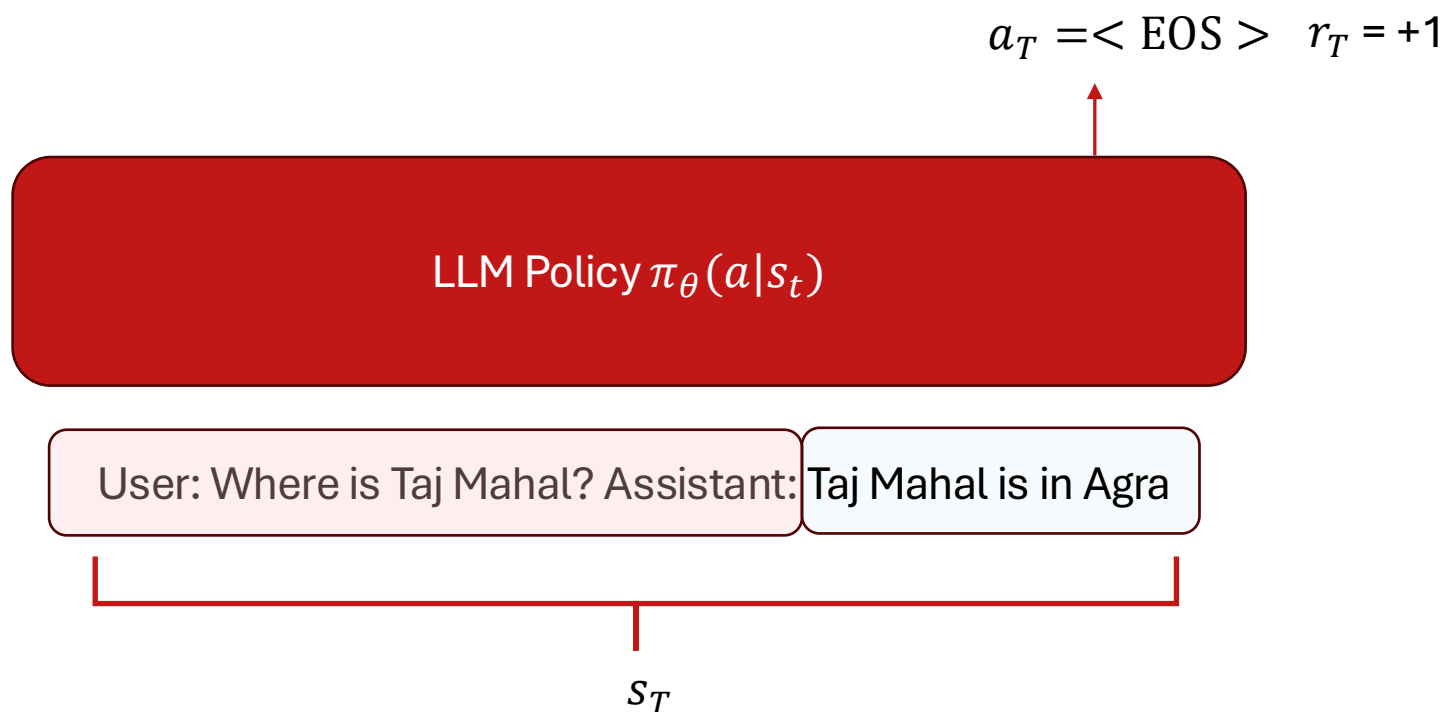
LLM as a Policy



LLM as a Policy



LLM as a Policy



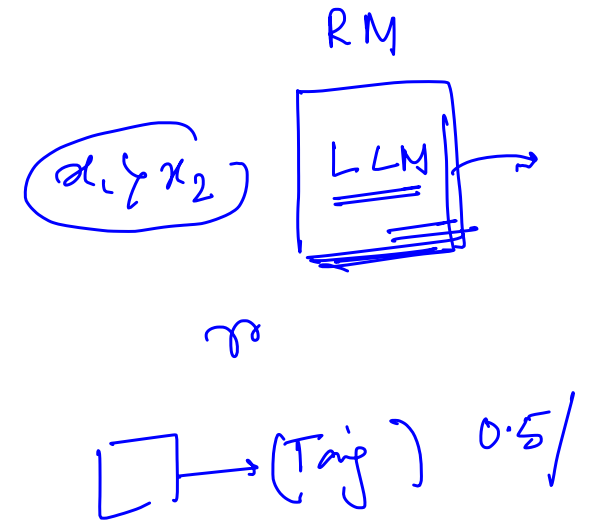
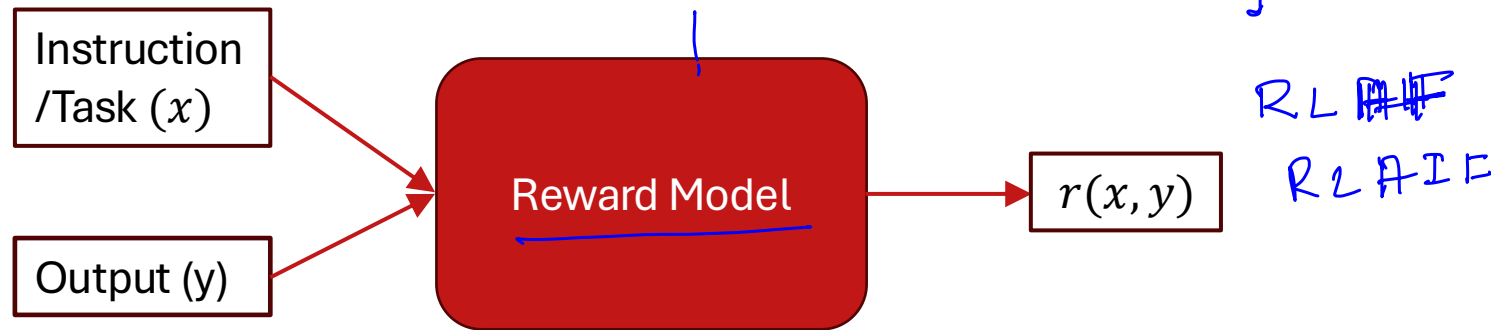
Who/What is the Reward Model?

- We can ask humans to give thumbs up/down to generated outputs and treat them as rewards.
- Challenges:
 - Human feedback is costly & slow.
 - Traditional RLHF (as we will see) requires constant feedback after every (few) updates to the model.
- Solution:
 - Lets train another LLM to behave like the reward model.



LLM as a Reward Model

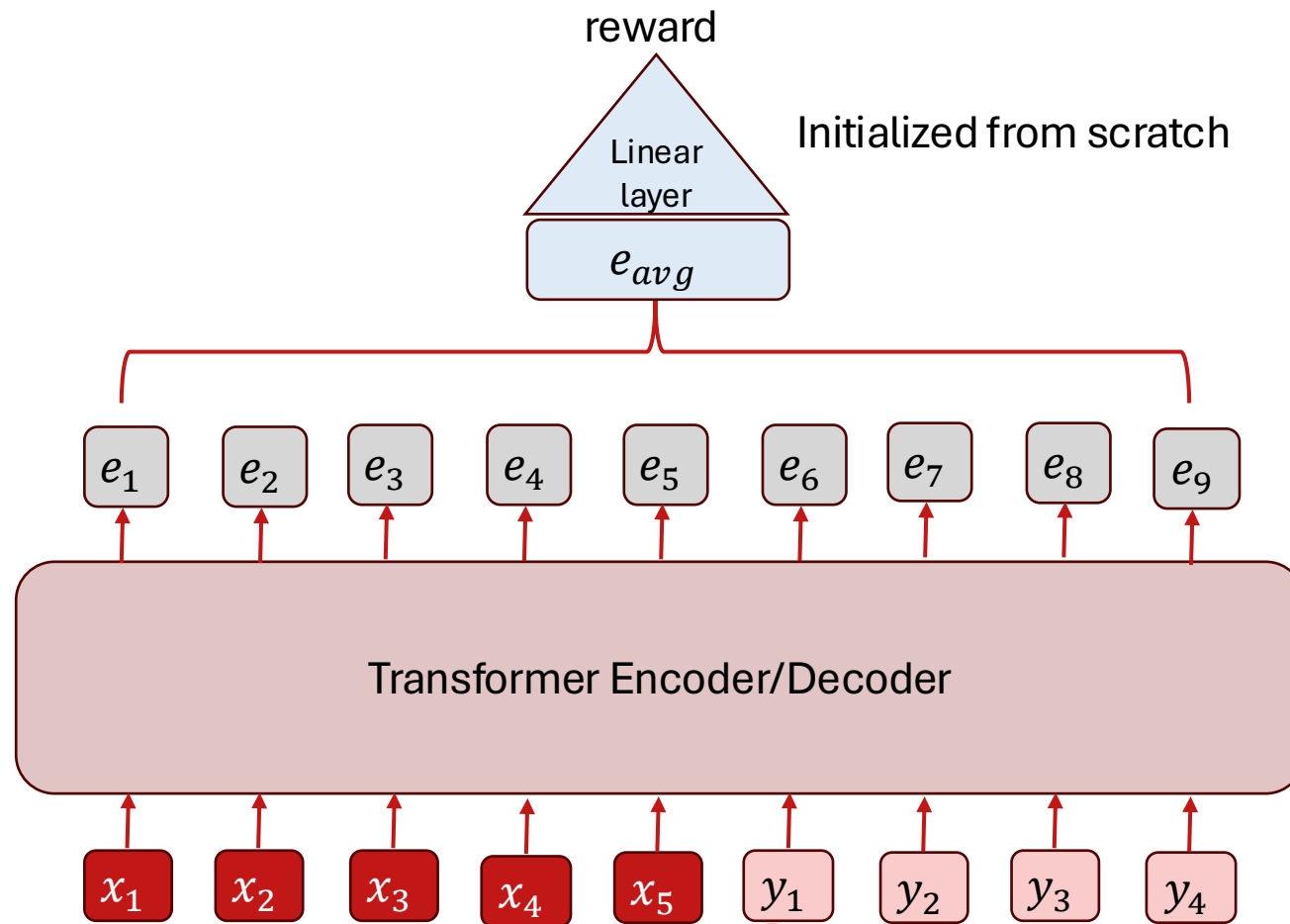
- Goal:



- Desirable: $r(x, y_1) > r(x, y_2)$ if y_1 is a better response than y_2
- If “better” is decided by humans, this pipeline is referred to as RLHF
- If “better” is decided by AI, it is called RLAIF



Architecture of the Reward Model



Training the Reward Model



An Intuitive View

$$\max_{\theta} \sum_{(x, y_+, y_-) \in D} \log \sigma(r_{\theta}(x, y_+) - r_{\theta}(x, y_-))$$

- Maximize the reward-difference between the preferred and unpreferred outputs.



Where Does the Data Come From?

- Prompts x
 - Can be sampled as a subset of instruction-tuning datasets.
 - ChatGPT used prompts submitted by humans for GPT-3
- Outputs y
 - Can be generated from an instruction-tuned LLM that you wish to align.
 - Can also be sampled from other LLMs to increase diversity.
- Preferences $y_+ > y_-$
 - Can be directly collected from humans (RLHF).
 - Another LLM can be tuned to judge (RLAIF).



The Reward Maximization Objective



The Objective

Given

- Base policy or reference policy $\pi_{ref}(y|x)$
 - Often, an instruction tuned LM that serves as the starting point of alignment
- Reward Model $r(x, y)$

Aim

- To find a policy $\pi_{\theta^*}(y|x)$
 - That generated outputs with high reward.
 - That stay close to the reference policy.



Why Care About Closeness to π_{ref} ?

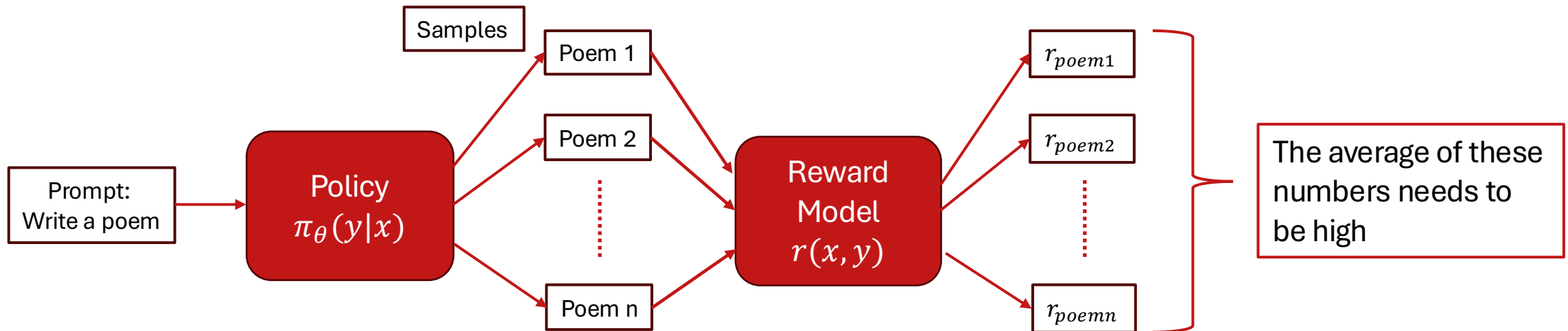
Reward Models are not perfect.

- They have been trained to score only selected natural language outputs.
 - Not the entire set of outputs for a given prompt
- The policy can hack the reward model – generate outputs with high reward but meaningless
- An input can have multiple correct outputs (Write a poem?)
 - Reward maximization can collapse the probability to 1 outputs
 - Staying close to π_{ref} can preserve diversity.



Formulating the Objective – Reward Maximization

- What does it mean for a policy to have high reward?



$\mathbb{E}_{y \sim \pi_\theta(y|x)} r(x, y)$ should be high

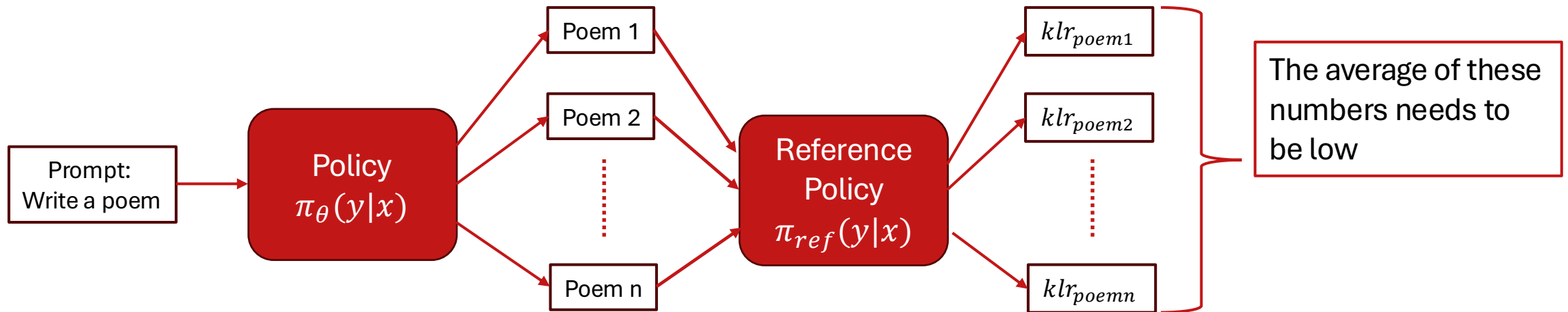


Formulating the Objective – Closeness to π_{ref}

- How do we capture closeness to π_{ref} ?

Policies are prob distribution

$$KL(\pi_{\theta}(y|x) || \pi_{ref}(y|x)) = \mathbb{E}_{y \sim \pi_{\theta}(y|x)} \left[\log \frac{\pi_{\theta}(y|x)}{\pi_{ref}(y|x)} \right]$$



Combining the Objective: Regularized Reward Maximization

- Maximize the reward

$$\mathbb{E}_{\pi_{\theta}(y|x)} r(x, y) \quad \uparrow$$

- Minimize the KL divergence

$$\mathbb{E}_{\pi_{\theta}(y|x)} \left[\log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} \right] \times \lambda \quad \downarrow$$

- Add a scaling factor β & combine

$$\mathbb{E}_{\pi_{\theta}(y|x)} \left[r(x, y) - \lambda \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} \right]$$

